

**Міністерство освіти і науки України  
Національний технічний університет України  
«Київський політехнічний інститут імені Ігоря Сікорського»  
Факультет інформатики та обчислювальної техніки  
Кафедра обчислювальної техніки**

**Лабораторна робота №1**

з дисципліни  
«Алгоритми і структури даних»

Виконала:

студентка групи ІМ-44  
Крошка Дар'я Олександрівна  
номер у списку групи: 11

Перевірів:

Сергієнко А. М.

Київ 2025

## Завдання:

Представити у програмі напрямлений і ненапрямлений графи з заданими параметрами:

- кількість вершин  $n$ ;
- розміщення вершин;
- матриця суміжності  $A$ .

## Визначення варіанту:

Номер групи складається з двох десяткових цифр:  $n_1n_2$ , а  $n_3n_4$  — це десяткові цифри номера варіанту, для двох попередніх робіт.

Мій варіант:

$$n_1n_2n_3n_4 = 4411$$

Кількість вершин  $n$  дорівнює  $10 + n_3$ .

Отже,  $10 + 1 = 11$  — маємо 11 вершин.

## Розміщення вершин:

- Вершини розміщуються колом, адже  $n_4 = 1$ .

Матриця суміжності:

Обчислюється коефіцієнт:  $k = 1.0 - n_3 \times 0.02 - n_4 \times 0.005 - 0.25$

Для  $n_3 = 1$  і  $n_4 = 1$  отримаємо:  $k = 1.0 - 1 \times 0.02 - 1 \times 0.005 - 0.25 = 0.725$

## Текст програми:

```
<!DOCTYPE html>
<html lang="uk">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title> Візуалізація графів </title>
  <style>
    body {
      display: flex;
      flex-direction: column;
      align-items: center;
      justify-content: center;
      height: 100vh;
```

```

        margin: 0;
        font-family: Arial, sans-serif;
        background-color: #E3F2FD;
    }
    .canvases {
        display: flex;
        gap: 20px;
        padding: 20px;
    }
    canvas {
        border: 3px solid #1565C0;
        background-color: #FFFFFF;
        border-radius: 10px;
    }
</style>
<script
src="https://cdnjs.cloudflare.com/ajax/libs/seedrandom/3.0.5/seedran
dom.min.js"></script>
</head>
<body>
    <h1 style="color: #0D47A1;">Візуалізація Графів</h1>
    <div class="canvases">
        <canvas id="neNapr" width="600" height="600"></canvas>
        <canvas id="napr" width="600" height="600"></canvas>
    </div>
    <script>
const canvasNeNapr = document.getElementById("neNapr");
const canvasNapr = document.getElementById("napr");
const ctxNeNapr = canvasNeNapr.getContext("2d");
const ctxNapr = canvasNapr.getContext("2d");

const n = 11; // 10 + n3 (where n3 = 1)
const radius = 20;
const centerX = 300, centerY = 300, circleRadius = 220;
const seed = 4411;
const n3 = 1, n4 = 1;

const k = 1.0 - n3 * 0.02 - n4 * 0.005 - 0.25;

const nodePositions = Array.from({ length: n }, (_, i) => {
    const angle = (i * 2 * Math.PI) / n;
    return {
        x: centerX + circleRadius * Math.cos(angle),
        y: centerY + circleRadius * Math.sin(angle)
    };
});

function generateAdjacencyMatrix(symmetric) {
    let matrix = [];

```

```

let rng = new Math.seedrandom(seed);
for (let i = 0; i < n; i++) {
  matrix[i] = [];
  for (let j = 0; j < n; j++) {
    let value = rng() * 2.0 * k;
    matrix[i][j] = value >= 1.0 ? 1 : 0;
  }
}
if (symmetric) {
  for (let i = 0; i < n; i++) {
    for (let j = i + 1; j < n; j++) {
      if (matrix[i][j] === 1) matrix[j][i] = 1;
    }
  }
}
return matrix;
}

function drawNodes(ctx) {
  nodePositions.forEach((pos, idx) => {
    ctx.fillStyle = "#1565C0";
    ctx.beginPath();
    ctx.arc(pos.x, pos.y, radius, 0, Math.PI * 2);
    ctx.fill();
    ctx.strokeStyle = "#0D47A1";
    ctx.lineWidth = 3;
    ctx.stroke();

    ctx.fillStyle = "white";
    ctx.font = "16px Arial";
    ctx.textAlign = "center";
    ctx.textBaseline = "middle";
    ctx.fillText(`${idx + 1}`, pos.x, pos.y);
  });
}

function drawArrow(ctx, x, y, angle) {
  const size = 10;
  ctx.save();
  ctx.translate(x, y);
  ctx.rotate(angle);
  ctx.fillStyle = "#0D47A1";
  ctx.beginPath();
  ctx.moveTo(0, 0);
  ctx.lineTo(-size, -size / 2);
  ctx.lineTo(-size, size / 2);
  ctx.closePath();
  ctx.fill();
  ctx.restore();
}

```

```

}

function drawEdges(ctx, matrix, directed) {
  ctx.strokeStyle = "#0D47A1";
  ctx.lineWidth = 2;
  ctx.globalAlpha = 0.85;
  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      if (matrix[i][j] === 1) {
        const { x: x1, y: y1 } = nodePositions[i];
        const { x: x2, y: y2 } = nodePositions[j];
        const angle = Math.atan2(y2 - y1, x2 - x1);

        const x1_new = x1 + radius * Math.cos(angle);
        const y1_new = y1 + radius * Math.sin(angle);
        const x2_new = x2 - radius * Math.cos(angle);
        const y2_new = y2 - radius * Math.sin(angle);

        ctx.beginPath();
        ctx.moveTo(x1_new, y1_new);
        ctx.lineTo(x2_new, y2_new);
        ctx.stroke();

        if (directed) {
          drawArrow(ctx, x2_new, y2_new, angle);
        }
      }
    }
  }
  ctx.globalAlpha = 1;
}

const matrixSymmetric = generateAdjacencyMatrix(true);
const matrixDirected = generateAdjacencyMatrix(false);

drawEdges(ctxNeNap, matrixSymmetric, false);
drawEdges(ctxNap, matrixDirected, true);
drawNodes(ctxNeNap);
drawNodes(ctxNap);

</script>
</body>
</html>

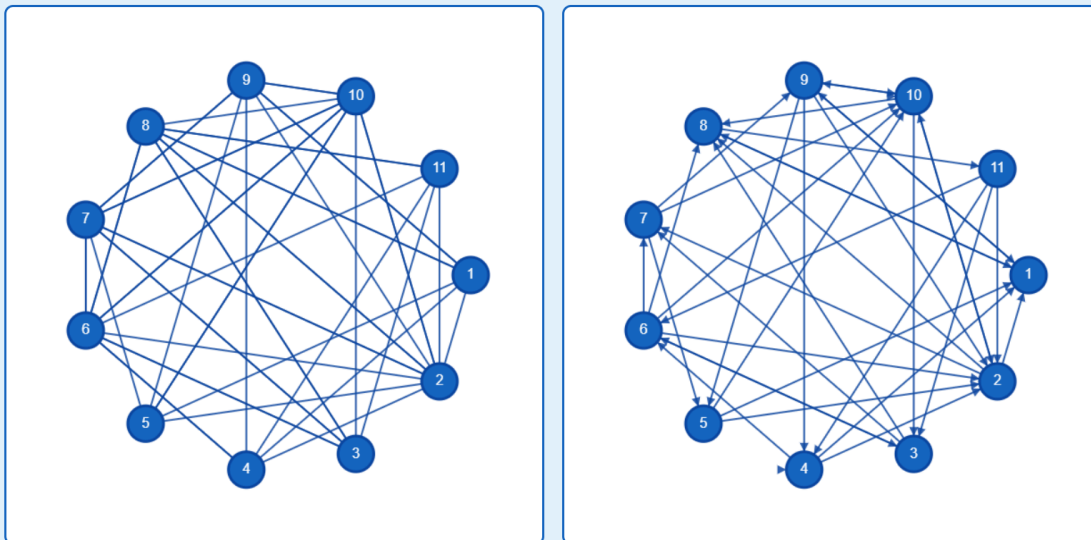
```

## Матриці:

```
▼ Array(11) ⓘ  
▶ 0: (11) [0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0]  
▶ 1: (11) [1, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0]  
▶ 2: (11) [0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0]  
▶ 3: (11) [1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0]  
▶ 4: (11) [1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0]  
▶ 5: (11) [0, 1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 0]  
▶ 6: (11) [0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1, 0]  
▶ 7: (11) [1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1]  
▶ 8: (11) [1, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 0]  
▶ 9: (11) [0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0]  
▶ 10: (11) [0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0]  
length: 11  
▶ [[Prototype]]: Array(0)  
  
▼ Array(11) ⓘ  
▶ 0: (11) [0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0]  
▶ 1: (11) [1, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0]  
▶ 2: (11) [0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0]  
▶ 3: (11) [1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0]  
▶ 4: (11) [1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0]  
▶ 5: (11) [0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 1, 0]  
▶ 6: (11) [0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0]  
▶ 7: (11) [1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1]  
▶ 8: (11) [1, 1, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0]  
▶ 9: (11) [0, 1, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0]  
▶ 10: (11) [0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0]  
length: 11  
▶ [[Prototype]]: Array(0)
```

## Візуалізація графів:

### Візуалізація Графів



## Висновки:

Лабораторна робота дозволила отримати практичні навички роботи з графами, їх представленням та візуалізацією, що є важливими для розуміння більш складних алгоритмів і структур даних. В процесі виконання було продемонстровано, як можна представити як направлені, так і ненаправлені графи за допомогою матриць суміжності, визначення параметрів графа, таких як кількість вершин та розміщення їх на колі,

дозволяє зрозуміти основи теорії графів і їх візуалізацію, використано HTML, CSS і JavaScript для візуалізації графів, що підтвердило важливість комбінування різних технологій для вирішення завдань у галузі інформатики та програмування.